

100071011 Computer Networks 2025-2026-2
Project-1
Reliable File Transfer using Go-Back-N protocol
Project Report

学号 (Student ID)	1120231863
姓名 (Name)	左逸龙
班号 (Class No.)	07112303
授课教师 (Instructor)	宿红毅

School of Computer
Beijing Institute of Technology
May 19, 2026

1. Requirement Analysis

The goal of this project is to implement reliable file transfer on top of UDP. UDP only provides datagram delivery and does not guarantee reliable delivery, ordering, duplicate suppression, or error recovery. Therefore, the reliability mechanism must be implemented in the application layer.

In this project, each UDP datagram carries one self-defined PDU. The sender divides a file into data blocks, attaches protocol fields and a CRC checksum, and sends the PDUs through a UDP socket. The receiver checks the PDU, accepts only the expected sequence number, writes correct data into the output file, and sends cumulative acknowledgements. When the sender does not receive the expected acknowledgement before timeout, it retransmits the unacknowledged PDUs according to the Go-Back-N protocol.

Requirement	Implementation	Evidence
Custom PDU	A fixed header and variable data field are defined.	gbn/pdu.py
CRC checksum	CRC-CCITT-FALSE is used to detect corrupted PDUs.	crc_ccitt_false()
UDP socket	One UDP datagram contains one encoded PDU.	socket.SOCK_DGRAM
Go-Back-N	The sender maintains a sliding window and retransmits outstanding PDUs after timeout.	gbn/host.py
Full duplex	Each host has sender and receiver threads.	run_demo.py
Loss and error simulation	The channel randomly drops or corrupts DATA/FIN PDUs.	gbn/channel.py
Configuration	Protocol and host parameters are read from JSON files.	configs/*.json
Logging and analysis	CSV logs are analyzed into summary tables and SVG charts.	gbn/analyzer.py

The expected output is a received file that is byte-by-byte identical to the original file. In the tests, this was verified by SHA-256 hash comparison.

2. Design

2.1. System Model

The system contains multiple hosts. Each host can send files to its configured peers and receive files from other peers at the same time. A sender thread reads the file, builds DATA and FIN PDUs, and sends them through an unreliable channel wrapper before the UDP socket. A receiver thread continuously receives UDP datagrams, decodes PDUs, checks CRC, updates the receive state, and sends ACK or FIN_ACK PDUs.

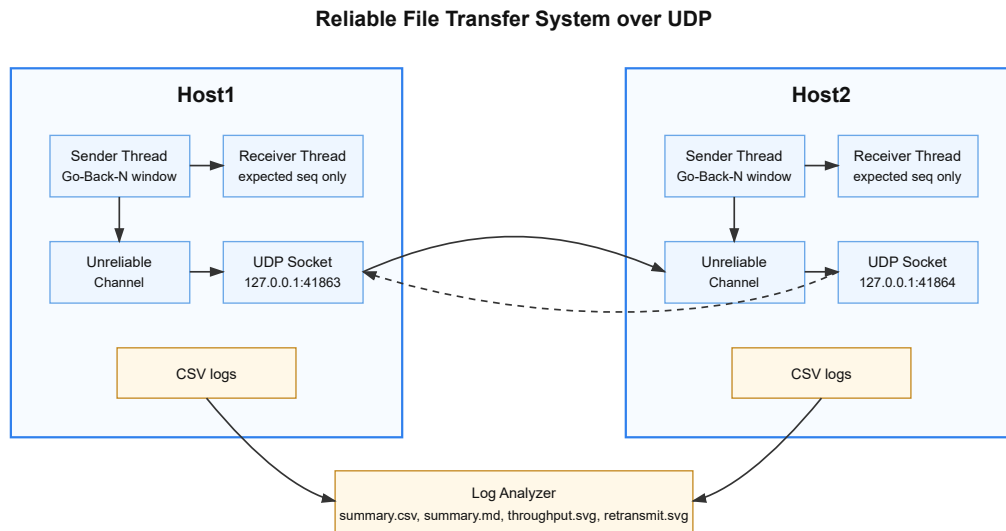


Figure 1: System architecture of the reliable file transfer system.

2.2. PDU Format

The PDU header is encoded with network byte order. Its fixed header size is 16 bytes. The data field is variable-length, but the implementation rejects data blocks larger than 4096 bytes.

Field	Size	Meaning
Magic	2 bytes	Protocol identifier, value 0x4742.
Version	1 byte	Protocol version, currently 1.
Type	1 byte	DATA, ACK, FIN, or FIN_ACK.
Session ID	4 bytes	Identifier of one file transfer session.
Seq	2 bytes	PDU sequence number in the configured sequence space.
Ack	2 bytes	Cumulative acknowledgement number.
Length	2 bytes	Length of the data field.
Checksum	2 bytes	CRC-CCITT-FALSE checksum.
Data	0-4096 bytes	File payload for DATA PDUs.

When encoding a PDU, the checksum field is first set to zero. Then CRC-CCITT-FALSE is calculated over the header and data. When decoding a PDU, the receiver recalculates the checksum with the checksum field zeroed. If the recalculated value is different from the received checksum, the PDU is treated as corrupted.

2.3. Go-Back-N Protocol

The sender stores all sent but unacknowledged PDUs in the current sending window. The variable `base` records the first unacknowledged absolute frame number, and `next_abs` records the next frame to send. ACKs are cumulative. When an ACK matches a sequence number in the current outstanding window, `base` moves forward.

The receiver window size is 1. The receiver only accepts the PDU whose sequence number equals the current expected sequence number. Out-of-order DATA PDUs are discarded and not buffered. The receiver acknowledges the last correctly received in-order PDU.

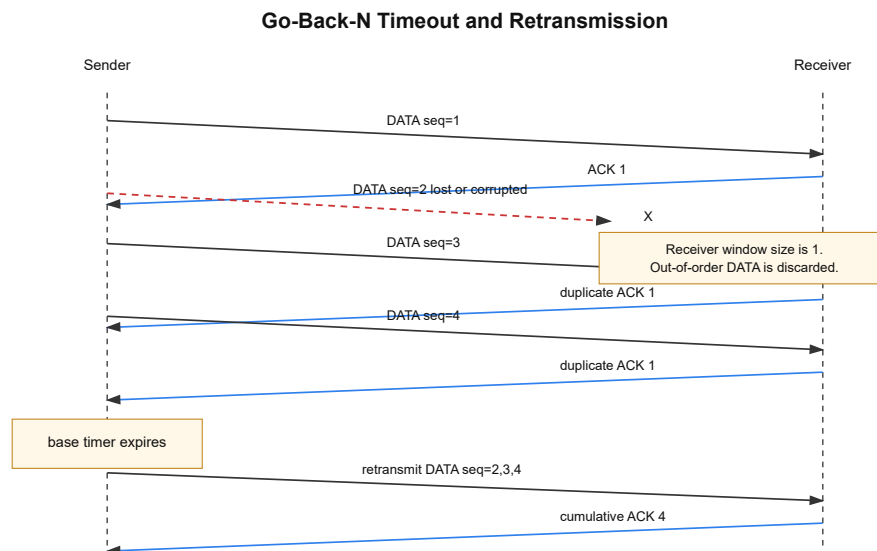


Figure 2: Go-Back-N retransmission after a lost or corrupted DATA PDU.

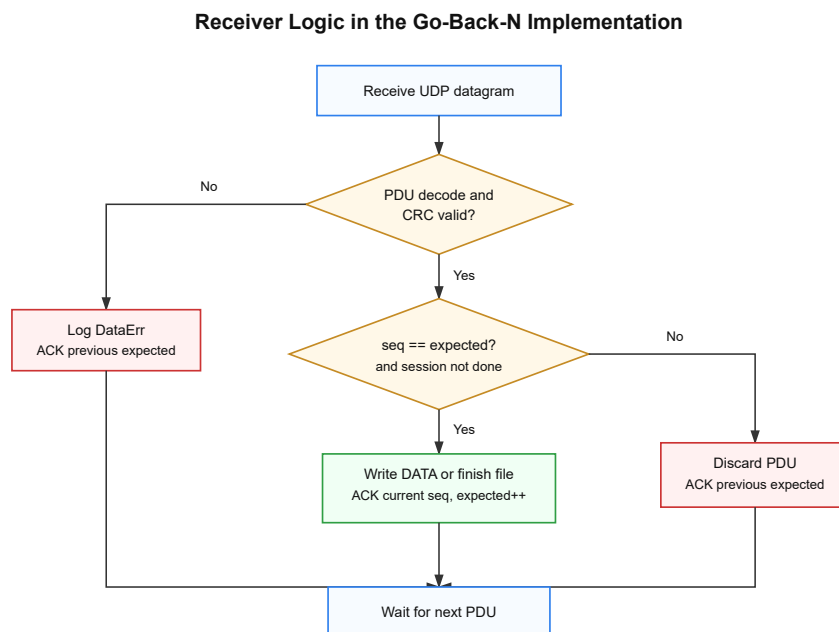


Figure 3: Receiver behavior with receive window size 1.

The sequence number in the PDU is a modular value. The implementation uses absolute frame numbers internally and writes $\text{absolute_number} \bmod 2^{\text{seq_bits}}$ into the PDU. The sending window size must satisfy $\text{sw_size} \leq 2^{\text{seq_bits}} - 1$. This check avoids ambiguity between old and new sequence numbers after wrap-around.

2.4. Configuration and Error Simulation

The main communication parameters are configured in JSON files. The default local ports are based on the last four digits of the student ID.

Parameter	Default value	Meaning
data_size	1024	Bytes of file data in one DATA PDU.
seq_bits	8	Sequence number space is 2^8 .

Parameter	Default value	Meaning
sw_size	8	Maximum number of outstanding PDUs.
init_seq_no	1	Initial sequence number used by both sides.
timeout_ms	300	Timeout value of the sender base timer.
lost_rate	0	Random DATA/FIN loss percentage.
error_rate	0	Random DATA/FIN corruption percentage.
local_port	41863/41864	UDP ports used by Host1 and Host2.

ACK and FIN_ACK PDUs are not randomly dropped or corrupted by default. This is a design choice used to focus the error simulation on data transfer. The report and tests therefore mainly analyze DATA/FIN loss and corruption. Another design note is that InitSeqNo is not negotiated by a handshake, so the sender and receiver configurations must use the same initial sequence number.

3. Development and Implementation

The project was implemented in Python using only the standard library. No database or third-party network library is required at runtime. The tested Python version was Python 3.13.5, while Python 3.10 or later is sufficient.

Path	Responsibility
gbn/pdu.py	PDU encoding, decoding, CRC, and sequence number utilities.
gbn/host.py	Host configuration, sender logic, receiver logic, ACK handling, and file hashing.
gbn/channel.py	Random packet loss and corruption simulation.
gbn/logging_utils.py	CSV logging of send and receive events.
gbn/analyzer.py	Log statistics, Markdown summary, and SVG charts.
run_demo.py	Starts Host1 and Host2 in one process for local full-duplex testing.
run_host.py	Starts one host from a JSON configuration file.
tests/test_pdu.py	Unit tests for CRC, PDU round trip, checksum failure, and sequence wrap-around.

The PDU implementation uses `struct.Struct("!HBBIHFFF")`. This format matches the header fields described in the design section. A corrupted PDU raises `PDUError` during decoding.

```
def crc_ccitt_false(data: bytes) -> int:
    crc = 0xFFFF
    for byte in data:
        crc ^= byte << 8
        for _ in range(8):
            if crc & 0x8000:
                crc = ((crc << 1) ^ 0x1021) & 0xFFFF
            else:
                crc = (crc << 1) & 0xFFFF
    return crc
```

The sender uses one base timer. If the timer expires, it logs a timeout event and retransmits all PDUs from `base` to `next_abs - 1`, which is the outstanding window.

```

if timer_started is not None and time.monotonic() - timer_started >= self.timeout:
    log.log(direction="send", event="timeout", status="T0", base=base,
next_seq=next_abs)
    for abs_no in range(base, next_abs):
        self._send_pdu(frames[abs_no], address, log, "T0", base, next_abs)
    timer_started = time.monotonic()

```

The CSV log contains enough information to reconstruct the transfer process.

Log field	Meaning
timestamp	Wall-clock timestamp of the event.
host_id, peer_id	Local host and peer host identifiers.
session_id	Transfer session identifier.
direction, event	Whether the row is send/receive and PDU/ACK/timeout.
pdu_type	DATA, ACK, FIN, or FIN_ACK.
seq, ack	PDU sequence number and acknowledgement number.
status	New, T0, OK, NoErr, or DataErr.
base, next_seq, expected_seq	Sender and receiver protocol states.
bytes, note	Payload length and channel note such as lost/error.

4. System Deployment, Startup, and Use

The source project directory contains a virtual environment and all required scripts. The following commands are run in the source directory.

```

python3 -m venv .venv
./venv/bin/python --version
./venv/bin/python generate_test_file.py data/host1.bin
./venv/bin/python generate_test_file.py data/host2.bin --seed 202

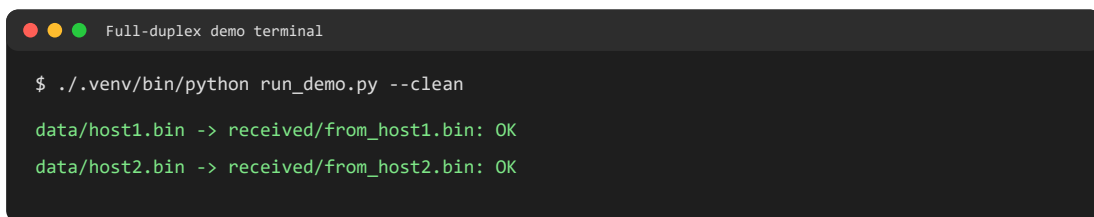
```

The local full-duplex demo starts Host1 and Host2 in one process. Host1 sends data/host1.bin to Host2, and Host2 sends data/host2.bin to Host1.

```

./venv/bin/python run_demo.py --clean
./venv/bin/python analyze_logs.py --log-dir logs --output-dir results

```



```

Full-duplex demo terminal

$ ./venv/bin/python run_demo.py --clean

data/host1.bin -> received/from_host1.bin: OK
data/host2.bin -> received/from_host2.bin: OK

```

Figure 4: Terminal output of the local full-duplex demo.

The hosts can also be started manually in two terminals.

```

./venv/bin/python run_host.py configs/host1.json
./venv/bin/python run_host.py configs/host2.json

```

```

configs/host1.json

{
  "host_id": "Host1",
  "local_ip": "127.0.0.1",
  "local_port": 41863,
  "data_size": 1024,
  "seq_bits": 8,
  "sw_size": 8,
  "init_seq_no": 1,
  "timeout_ms": 300,
  "error_rate": 0,
  "lost_rate": 0,
  "peers": [
    { "peer_id": "Host2", "peer_port": 41864,
      "send_file": "data/host1.bin",
      "receive_file": "received/from_host2.bin" }
  ]
}

```

Figure 5: Key fields in the Host1 configuration file.

After transfer, logs are written to logs/, received files are written to received/, and statistical outputs are written to results/.

5. System Test

5.1. Unit Tests

The unit tests check the CRC standard vector, PDU encoding and decoding, checksum failure detection, and sequence number wrap-around window logic.

`PYTHONPATH=. ./venv/bin/python -m unittest discover -s tests -p "test_*.py"`

```

Unit test terminal

$ PYTHONPATH=. ./venv/bin/python -m unittest discover -s tests -p 'test_*.py'

...
-----
Ran 3 tests in 0.000s

OK

```

Figure 6: Unit test result for PDU, CRC, and sequence number utilities.

5.2. Integrated Transfer Test

The default demo transferred two files larger than 3 MB at the same time. The output files were verified by SHA-256.

```

SHA-256 verification terminal

$ sha256sum data/host1.bin received/from_host1.bin data/host2.bin received/from_host2.bin

dcd36f1582dd0b72f11130426a20baa6f31c7f1a742dca07b6021a5215512939 data/host1.bin
dcd36f1582dd0b72f11130426a20baa6f31c7f1a742dca07b6021a5215512939 received/from_host1.bin

35dd3c26d3239f0891efc44b39632cd01ce45ca36022189519b200d4705132b5 data/host2.bin
35dd3c26d3239f0891efc44b39632cd01ce45ca36022189519b200d4705132b5 received/from_host2.bin

```

Figure 7: SHA-256 comparison between original and received files.

Test case	Configuration	Actual result	Status
Unit tests	Default test suite	Ran 3 tests, 0K	Passed
Full-duplex transfer	Host1 <-> Host2	Both demo outputs were 0K.	Passed
File integrity	Two 3 MB+ files	Both SHA-256 pairs matched.	Passed
Packet loss	lost_rate=5, error_rate=0	retransmits=2463, timeouts=308	Passed
Packet corruption	lost_rate=0, error_rate=5	DataErr=430	Passed
Loss and corruption	lost_rate=5, error_rate=5	retransmits=5440, timeouts=680, DataErr=556	Passed
Maximum data size	data_size=4096	Transfer passed.	Passed
Invalid data size	data_size=4097	ValueError was raised.	Passed
Sequence wrap-around	seq_bits=3, sw_size=7	Transfer passed with wrap-around.	Passed
Invalid window size	seq_bits=3, sw_size=8	Startup was rejected.	Passed
Multi-host transfer	Host1 to Host2/3/4	All received files matched Host1 source file.	Passed

The following sender log excerpt shows the main Go-Back-N behavior. The sender first sends eight new DATA PDUs. After timeout, it retransmits the whole outstanding window.

Event	Type	Seq	Status	Base	Next
pdu	DATA	1	New	0	1
pdu	DATA	2	New	0	2
pdu	DATA	8	New	0	8
timeout			T0	0	8
pdu	DATA	1	T0	0	8
pdu	DATA	2	T0	0	8
pdu	DATA	3	T0	0	8

6. Performance and Analysis

The default full-duplex demo generated four CSV log files: two sender logs and two receiver logs. Throughput is meaningful for receiver logs because successfully received bytes are counted on the receiving side. Sender logs have bytes_ok=0 in the analyzer output, so they should not be interpreted as zero real transfer throughput.

Receiver log	Duration	Bytes OK	Throughput
Host1_recv_Host2_ec50f047.csv	17.046089 s	3145851	180.22 KiB/s
Host2_recv_Host1_f4dd0758.csv	16.816290 s	3145851	182.69 KiB/s

Receiver Throughput (KiB/s)

Figure 8: Receiver-side throughput in the default full-duplex demo.

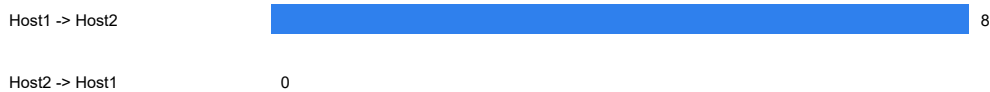
Retransmitted PDU Count in Default Demo

Figure 9: Retransmitted PDU count in the default full-duplex demo.

The error injection tests show the expected behavior of Go-Back-N. Packet loss causes timeouts and retransmissions. Packet corruption is detected by CRC and recorded as DataErr. When both loss and corruption are enabled, the number of retransmissions increases because one lost or corrupted PDU can force retransmission of multiple outstanding PDUs.

Scenario	Loss	Error	Retransmits	Timeouts	DataErr
Loss only	5%	0%	2463	308	0
Corruption only	0%	5%	-	-	430
Loss and corruption	5%	5%	5440	680	556

These stress tests were executed on a copied project directory during code checking, so the submitted source directory was not polluted by temporary configurations. In all tested cases, the received files matched the original files.

7. Summary or Conclusions

This project implemented reliable file transfer over UDP using the Go-Back-N protocol. The implementation includes a self-defined PDU format, CRC-CCITT-FALSE checksum, sliding-window sending, cumulative acknowledgements, timeout retransmission, receiver-side in-order delivery, full-duplex transfer, configurable packet loss and corruption, CSV logging, and log analysis.

The tests show that the implementation can transfer files larger than 3 MB correctly. SHA-256 comparison proved that the received files were identical to the original files. Additional tests with packet loss, packet corruption, boundary data sizes, sequence number wrap-around, invalid parameters, and multi-host transfer also passed.

The main limitation is that `InitSeqNo` must be configured consistently on both sides because there is no handshake to negotiate it. Also, ACK and FIN_ACK PDUs are not randomly lost or corrupted by default. These choices simplify the project and make the DATA transfer behavior easier to analyze.

8. References

1. Andrew S. Tanenbaum and David J. Wetherall, *Computer Networks*, 5th Edition, Prentice Hall, 2011.
2. Python Software Foundation, “socket – Low-level networking interface”, Python Documentation.
3. Python Software Foundation, “struct – Interpret bytes as packed binary data”, Python Documentation.
4. Python Software Foundation, “threading – Thread-based parallelism”, Python Documentation.

5. RevEng CRC Catalogue, “CRC-16/IBM-3740”, a CRC-CCITT-FALSE compatible parameter set.
6. Typst Documentation, “Typst Reference”.

9. Comments

This project helped me understand why reliability cannot be assumed when UDP is used. Implementing Go-Back-N also made the relationship between sequence numbers, cumulative ACKs, timeout retransmission, and receiver-side ordering clearer. The logging and analysis part was useful because it showed the cost of reliability under packet loss and packet corruption.